PYTHON

This guide provides a straightforward and efficient guide for mastering the basic concepts of programming from beginning. AI has been used in some parts to enhance understanding of a concept.

INTERPRETER: A computer program that runs code without having to be compiled into machine code first. The code is executed line by line. Although interpreters generally operate more slowly than compiled code, they do enable more rapid testing and debugging of programming.

Example:

```
print("Hello, world!")
```

When the above code is executed, the Python interpreter reads the print("Hello, world!") line and immediately performs the action of displaying "Hello, world!" on the screen.

COMPILER: Is a software that translates source code written in a high-level programming language into machine code that a computer's processor can execute. Compiled programs usually run faster than interpreted ones because the translation is done before execution.

Example:

```
#include <stdio.h>

int main() {
   printf("Hello, world!\n");
   return 0;
}
```

When the above code is compiled with a C compiler, the compiler translates the entire program into machine code, producing an executable file that anyone can run. Anyone can then run this file to see "Hello, world!" printed on the screen.

INTEGRATED DEVELOPMENT ENVIRONMENT (IDE) application that comes with every tool required to create software. Usually, it includes a debugger, compiler/interpreter, code editor (where the code is written), and many more tools.

Example:

VS Code, Eclipse, PyCharm

CONSOLE (or command line interface, CLI) is a text-based interface that allows users to interact with the operating system by typing commands. It is used for running programs, navigating the file system, managing processes, and performing administrative tasks. Developers often use the console to execute operating system tools, commands, and scripts.

Modern IDEs allow users to perform command line operations directly within the IDE, seamlessly integrating the console into the application.

PRINT FUNCTION

'print' in Python is a built-in function used to output text or other data to the console. It is versatile and can handle various types of data, including strings, numbers, and objects.

Example:

```
print("Hello, World!") #Strings always surrounded with either double or single quotes.

print(123) #Printing numbers does not require any special character.

my_list = [1, 2, 3]
print(my_list) #Object can be printed by including the name of the object in the print statement.
```

To modify the appearance and behaviour of the text output special characters can be used. Here are some commonly used control characters in Python:

\n: Newline character, used to initiate a new line within a string.

```
print("Hello\nWorld")          Output: Hello
                                       World
```

\t: Tab character, used to insert a space in a string.

```
print("Column 1\tColumn 2")    Output: Column 1     Column 2
```

sep and **end** keywords are further used to enhance the print statement functionality.

sep: Determines what separator should be used between the values that are passed to the print() function. By default, sep is set to a single space ' '.

Example:

```
print('Hello', 'world', sep=', ')     Output: Hello, world
print(1, 2, 3, sep=' - ')             Output: 1 - 2 - 3
```

end: By default, when you use print() without specifying end, Python adds a newline character ('\n') at the end of the printed output. This newline character causes the next print() statement to start on a new line.
However, to modify this behaviour 'end' can be used. When end is specified in a print() statement, Python will append to any character or string provided to the end of the printed output instead of the default newline ('\n').

Example:

```
print('Hello', end=' ')
print('world')        Output: Hello world

print("Hello", end='--')
print("World!")       Output: Hello--World!
```

VARIABLE

Variables are places to store information (VALUES) such as: Numbers, Text, Lists, and so on. We can look at it as a LABEL. Each variable has a name, we can use the data stored in it when in need.

Variable

Fred = 100 ← Value stored in the variable called Fred.

Equal sign used to create a variable.

Cannot start a variable with number or special character. Cannot use any other special character other than Underscore. No variables name can be the same as one of Python keywords.

age ✓	age_ ✓
Age ✓	_age ✓
1Age ✗	age_* ✗

A simple way to check if variable name is a keyword:

Enter desired variable

```
if "exit" in dir(__builtins__):
    print('Yes')
else:
    print("No")
```

Different types of data can be stored in a variable. There are **four** main types of data Python knows: String, Integer, Floating Point, and Boolean.

STRING (str) Strings are essential tools for handling text-based information and performing diverse tasks within software development. **Strings are always enclosed within quotation marks (either single or double. Mathematical operations cannot be performed on strings.**

('text') ✓	("text") ✓
("text') ✗	('text") ✗

Example:

```
#Define a string variable.
string = "Hello, world!"
#Print the value of variable to the console.
print(string)
```

Output: Hello, world!

There are several built-in functions specifically designed for string manipulation.

String manipulation in Python is essential for a wide range of tasks, including searching, replacing, splitting, and joining strings. These techniques facilitate data validation, conversion, and presentation, enhancing the functionality and usability of Python programs.

.upper(): This method converts all the characters in a string to uppercase.

```
text = "Hello, World!"
uppercase_text = text.upper()
print(uppercase_text)
```

Output: HELLO, WORLD!

.lower(): This method converts all the characters in a string to lowercase.

```
text = "HELLO, World!"
```

```
lowercase_text = text.lower()
print(lowercase_text)              Output: hello, world!"

membership=['gold','silver']
user_membership = input("Enter your membership level: ").lower()
```

.lower() and .upper() disregards the distinction between uppercase and lowercase letters. Therefore, if someone enters "gold," "Gold," or "gOlD," they will all be treated equally and considered valid if "gold" is included in the list of choices.

.split(): Method in Python, used to break a string into a list based on a specified delimiter (specific character). It is commonly used to break a sentence or a CSV (Comma-Separated Values) line into individual words or fields.

Example 1:
In this example below the sentence is converted into a list using the space delimiter, the value of name and age are then extracted and stored in the appropriate variables.

```
sentence = " My name is Sandeep and I am 27 years old."

words = sentence.split(“ “) #Split using space as a delimiter.
name=words[3]
age=words[7]

print(“Name:”, name)                Output:  Name: Sandeep
print(“Age:”, age)                  Output:  Age: 27
```

Example 2:

```
csv_data = "Alice,30,New York"
fields = csv_data.split(",") #Split using a comma as the delimiter.
print(fields)                       Output:  ['Alice', '30', 'New York']
```

String Concatenation:
It is the process of merging multiple strings into a single string. This operation is commonly performed using the "+" operator in programming languages. The “+” is used to easily combine different words or messages to create new ones.

```
str1 = "Hello"
str2 = "World"
result = str1 + str2
print(result)                       Output:  HelloWorld
```

F-String Formatting

F-strings are a quick and easy way to put variables or calculations directly into strings in Python, making the code shorter and more readable.

SYNTAX:

Start the string literal with the **f** or **F** before the opening quote.

variable = f" The sum of {x} and {y} is {x + y}, and the product is {x * y}"

{ }: Can hold values stored in **variables** or can do actual **calculation** performed on the variables.

Example 1:

```
x = 5
y = 10
result = f"The sum of {x} and {y} is {x + y}, and the product is {x * y}."
print(result)
```

Output: The sum of 5 and 10 is 15, and the product is 50.

Example 2:

```
#Create a function for the operation.
def double(x):
    return x * 2

num = 5
result = f"The double of {num} is {double(num)}."
print(result)
```

Output: The double of 5 is 10.

Example 3:

```
numbers = [1, 2, 3, 4, 5]
print(f"The first number is {numbers[0]}")
```

Output: 0

INTEGER (int) is a whole number without a decimal point.

Integers are utilized for various calculations, including mathematical operations, searching for items in a list, looping, decision-making, and comparisons. Additionally, integers generally use less computer memory compared to numbers with decimal points, making them efficient for storage and computation.

Variable. The value of this parenthesis will be converted into integer.

year = int(" ")

Followed by equal.

Example 1:

age = 23

The value 23 is not a string; it's an integer. In Python, integers are represented directly without quotation marks.

Example 2:

```
#Created variables length and width, each assigned to a string value representing the length and width of a rectangle,
length = "10"
width = "5"

#Converting strings to integers
length1 = int(length)
width1 = int(width)
#Calculating the area of the rectangle.
area = length1 * width1
# Printing the result
print("Area is:", area)
```

Output: Area is: 50

Converting strings to integers is necessarywhen working with data that is initially stored as text but requires numerical operations or comparisons.

FLOATING POINT (float) are a data type used in programming to represent numbers that have a decimal point. They are essential for mathematical operations involving fractions or real numbers. Floating points values have limited range and limited precision.

The value of this parenthesis will be converted into decimal point .

num= float (" ")

Example 1:

```
#Define an integer variable named.
value = 23
#Convert the integer value to a float and assign it to the variable value2.
value2 = float(value)
#Print the value of value2, which is now a floating-point representation of the original integer value.
print(value2)
```

Output: 23.0

To format the values of decimal points use the FORMAT FUNCTION. Formatted numerical values are useful when displaying results in which only a certain number of decimal place need to be displayed.

Example 2:

```
tax=12243.564
print('Your tax: ' , format (tax, '.2f')
```

Output: Your tax: 12243.56

```
print('Your tax: ' , format (tax, ',.2f')
```

Output: Your tax: 12,243.56

.2f rounds the value to 2 decimal places. ,.2f rounds the value to 2 decimal places adding a comma separator to the result.

BOOLEAN (bool) Boolean values are a fundamental data type in programming that can only take on one of two possible values: True or False.These values are often used in conditions and logical expressions to control the flow of a program.

Example:

```
#Define a boolean variable named is_raining and assign it the value True
is_raining = True

#Check if it's raining
if is_raining:
    print("Remember to bring an umbrella!")
else:
    print("No need for an umbrella today.")
```

DOCSTRING

Docstrings are special strings used clarify the code, helping anyone reading the code understand its purpose, parameters, and what it returns. They are enclosed within triple quotes (''' ''' or """ """) and can be used for functions, classes, methods, and modules.

Additionally, the text can be saved in variables and used later in the code. This is helpful for holding huge text blocks as variables or for providing context for the code.
The stored content can be manipulated like any other string variable, and it will retain all of its formatting, including newline characters.

EXAMPLE:

```
#Storing a multi-line string using triple quotes in a variable
doc_variable = """
This is a multi-line string.
It can span multiple lines.
It retains all formatting, including
line breaks and indentation.
"""
print(doc_variable)

#Adding a new line to the end of multi-line string.
doc_variable += "This is a new line added to the multi-line string.\n"
print(doc_variable)
```

PYTHON OPERATORS

Operators in Python are special symbols that perform operations on variables and values. They can be categorized into several types:

- **Comparison Operators compare values and return a Boolean value.**

  ==: Returns True if left value and right value are SAME, False otherwise.

  !=: Returns True if left value and right value are NOT EQUAL, False otherwise.

  >: Returns True if left value is BIGGER THAN right value, False otherwise.

  <: Returns True if left value is SMALLER THAN right value, False otherwise.

  >=: Returns True if left value is BIGGER THAN OR EQUAL to the right value, False otherwise.

  <=: Returns True if left value is SMALLER THAN OR EQUAL to the right value, False otherwise.

- **Logical Operators combine or modify Boolean expressions to make a decision**

  and Operator: Returns True if both conditions are True, and False otherwise.

  or Operator: Returns True if at least one condition is True, and False if both are False.

  not Operator: Returns the opposite value of the element.

  Example:

```
age = 25
income = 40000
if age < 30 and income > 30000:
    print("You are young and have a good income")
else:
    print("You may not be young or your income may not be high enough")
```

- **Membership Operators used to examine whether a particular element is present within a given sequence, such as a string, list, tuple, or set.**

  in operator returns True if a specified value is found in the sequence and False otherwise.

  not in operator returns True if a specified value is not found in the sequence and False otherwise.

  Example:

```
my_list = [1, 2, 3, 4, 5]
if 3 in my_list:
    print("3 is present in the list.")
```

INPUT

Is used to store data from the user, also called PROMPT (request input from user).
The input() function in Python always returns the user's input as a string, regardless of whether the input is a number or text.
To treat the input as a numerical value (either an integer or a float), you must clearly convert it using functions such as int() or float()

Syntax:

Create variable that will store the value from user
Prompt user to enter data.
String that serves as the prompt displayed to the user.

```
name = input ('What is your name?:')
print('My name is:' , name) #Output will be the name typed by the user.
```

- x = int(input('What is your age? '))

The result from input() is passed to the int() function, which converts the user's input (which is initially a string) into an integer. This is necessary if you want to perform mathematical operations or comparisons. The resulting integer from the conversion is assigned to the variable x. Now, x holds the user's age as an integer, and can be used to do more operation.

INDENTATION

In Python, indentation is vital for code structure, readability and defining blocks of code. Block of code refers to a group of statements that are treated as a unit, because they are logically related and should be executed together under certain conditions, such as within loops, conditional statements (like if-else statements), or function definitions. Indentation persists until an unindented line is encountered, indicating the end of the block.

Example:

```
x = 5
#Start of the if-else block
if x > 0:
    print("x is positive")  #Indented block for if condition
else:
    print("x is non-positive")  #Indented block for else condition
```

IF-ELIF-ELSE STATEMENTS

Selection control statements, such as if-elif-else statements, enable a program to make decisions based on specific conditions. They execute particular blocks of code depending on whether these conditions evaluate to True or False. These statements rely on various expressions like comparison operators (e.g. ==, !=), logical operators (e.g. and, or, not), membership operators (e.g. in, not in), and identity operators (e.g. is, is not) to evaluate conditions. By utilizing if-elif-else statements, developers can direct the flow of a program effectively, allowing for the execution of distinct code blocks based on different conditions.

```
if condition1:
    #The IF statement is always the first condition.
    #Indented block of code will be executed if condition1 is TRUE.
elif condition2:
    #Can have multiple elif conditions.
    #Indented block of code will run only if condition1 is False and condition2 is True.
else:
    #Always the last condition.
    #Else statement can be used even if there is no elif statement present.
    #Indented block of code will run only if both condition1 and condition2 are False.
```

Once the code inside a block of conditions is evaluated as True, the program exits the entire if-elif-else structure, does not check any further conditions, and moves on to the next line of code outside the conditional block.

Syntax:

```
temp=40
 Evaluating Condition
if temp >= 40 :  ←COLON After if – elif – else statements
    print("Hot Weather")
```

In Python, indentation following conditions signifies that the subsequent block of code is associated with that condition. The indentation persists until an unintended line is encountered, indicating the end of the block.

Example:

```
#Create a list of valid membership levels
membership_list = ['gold', 'silver', 'bronze']
#Ask the user for input and convert it to lowercase, so that input is case-sensitive
user_membership = input("Enter your membership level: ").lower()

#Check if the user's input is in the list of valid membership levels.
if user_membership in membership_list:
    #If statement TRUE
    print("You have a valid membership level.")
#If all of the above statements are FALSE.
else:
    print("Invalid membership level.")
```

LOOPS

Loops in Python are control flow statements that allow to execute a block of code repeatedly based on a condition or a set of conditions.
Python provides two primary types of loops: **WHILE** and **FOR** loop

While loop allows to repeatedly executing a block of code if a specified condition is true. The loop will continue to run as long as the condition remains true and will terminate as soon as the condition becomes false.
This structure is essential for automating tasks that require repeated execution until certain conditions are met or changed.

The count value is essential for the loop to function correctly. It sets the starting point for the loop's iterations. With each iteration of the loop, count is incremented (count += 1), updating its initial value based on the loop's logic until the loop's exit condition is met. So, the initial value of count directly influences the loop's operation and determines how many times the loop will execute.

```
count = 0
while count <= 3:
    print(count)
    count += 1
```

This is the body of the while loop, which will be executed repeatedly as long as the while condition evaluates to true.

Indented block of code immediately following the while statement is the body of the loop.

ITERATION	COUNT	COUNT<=3
1	0	TRUE
2	1	TRUE
3	2	TRUE
4	3	FALSE Loop Terminate.

Example 1:

```
#Initializing user_input with a starting value provides the while loop with an initial condition to evaluate, enabling the loop to either begin its first iteration or not, based on the given condition.
user_input = " "

#The loop body continues executing until the user enters the "quit" word. When user enter “quit” program will exit loop going to the next statement.
while user_input.lower() != "quit":
    #Prompt the user to enter a word
    user_input = input("Enter a word (type 'quit' to exit): ")
    # Display the user's input
    print("You entered:", user_input)

#Not indented because it is meant to execute after the while loop exits.
print("Program ended.
```

Example 2:

```
# Welcome message
print("Welcome to the Word Guessing Game! \nTry to guess the secret word. You have 3 attempts ")

# Define the secret word
secret_word = "python"
# Initialize variables
guess = ""
attempts = 0
max_attempts = 3

# Main game loop, the main game loop will continue running as long as the condition specified in the while statement remains true.
while attempts < max_attempts:
    # Prompt the user to enter a guess
    guess = input("Enter your guess: ")
    # Increment the number of attempts
    attempts += 1

    # Check the guess
    if guess == secret_word:
        print("Congratulations! You guessed the secret word correctly.")
        break  # This break statement exits the while loop
    elif attempts < max_attempts:
        print("Incorrect guess. Try again.")
    else:
        print(f'Sorry, you've used all {max_attempts} attempts. The secret word was '{secret_word}'.")
```

Example 3:

```
#The while True statement in programming creates an infinite loop, which means the loop will keep running forever or until an explicit instruction to stop is encountered within the loop body.
while True:
    user_input = input("Enter 'exit' to quit: ")
    if user_input.lower() == 'exit':
        print("Exiting the loop.")
        break  #This exits the loop
```

For loop is another type of iterative control statement in Python. It allows executing a block of code repeatedly for a fixed number of times or over a sequence of items such as lists, tuples, dictionaries, or strings.

A for loop in Python allows to address each item independently, processing them one at a time in a sequential manner; lines are processed one after another, maintaining the order in which they appear in the file.
During each iteration, the loop assigns the current item to a variable, allowing for manipulation or analysis within the loop. As the loop iterates through the sequence, it continues until it has processed all items, at which point the loop terminates, and the program proceeds to the next statement following the loop.

```
my_list=[1,2,3]
for number in my_list:
```

number is the **variable** that will hold one item at a time from the list my_list, moving to the next item after the current one is processed. The code block inside the loop is then executed using the value of number.

Example 1:

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
   if num >= 4:
      print(num)
```

Output:4
5

Example 2:

```
word = “Sandeep”
characters_to_match = [‘a’, ‘e’, ‘l’]
for letter in word:
   if letter in characters_to_match:
      print(letter,end=” “)
```

Output: a
e
e

CONTROL FLOW STATEMENTS

To manipulate the flow within loops, **continue, break**, and **return** statements are used for more advanced data processing. These control statements allow for skipping iterations, exiting loops early, and returning from functions, enabling more flexible and efficient handling of data.

The **continue** statement is used within loops. When encountered, continue immediately ends the current iteration of the loop and jumps back to the top of the loop for the next iteration, skipping the remaining code in the current iteration.

Example 1:

```
for i in range(1, 6):
    if i == 3:
        continue #Skip the rest of the loop for i == 3
    print(i) #This will print 1, 2, 4, 5
```

Example 2:

```
values = [-1, 2, -3, 4, 5]
for value in values:
    if value < 0:
        continue # Skip negative numbers
    print(value) #This will print 2, 4, 5
```

The **break** statement is used to exit a loop prematurely. When Python encounters a break, it immediately stops the execution of the current loop and exits out of it. The break statement only affects the innermost loop it is in and does not terminate the execution of the entire program or any enclosing loops.

Example:

```
for i in range(1, 10):
    if i == 5:
        break  #Exit the loop when i is 5
    print(i)
#This will print numbers 1,2,3,4 and then stop.
```

The **return** statement is used exclusively in functions. It allows a function to send a result back to the code that called it. This result can be of any data type, such as a number, string, list, or object. The return statement is particularly useful for processing data, performing calculations, and then providing the results for further use.

When a return statement is executed, the function immediately stops running, meaning any code that follows the return statement within that function will not be executed.

Example 1:

```
def calculate_average(numbers):
    if not numbers:  #Check if the list is empty
        return None
    return sum(numbers) / len(numbers)  #Calculate and return the average

scores = [10, 20, 30, 40, 50]
average_score = calculate_average(scores)
print(f"The average score is: {average_score}"
```

Example 2:

```
def square(x):
    return x * x  # Return the square of x

result = int(input('Enter a number: '))  # Get user input and convert it to an integer
num = square(result)  # Call the square function with the user input
print(num)  # Print the squared result
```

LISTS

Lists are used to store multiple items in a single variable, can contain all sort of data: numbers or strings. Lists items can be ordered, changed, and allow duplicate values.
Lists are typically indexed data structures.
Lists, tuples, and many other data structures in Python support indexing, allowing programmers to access specific elements within these data structures, facilitating efficient manipulation and retrieval of data.

0 1 2 3 ← List starts at index position 0.
list= [‘a’, ’b’, ’c’, 1]
Created, finished with square brackets.

To print 1st value: print(list [0]) Output: a
To print values from index position one up to index position three (not included):
print(list [2 : 3]) Output: [‘b’ , ‘c’]
Not included.
Included

Nested list

A nested list is a list that has other lists as its elements. List 0 and List 1 are nested lists. They are the elements of the outer list List, which contains two inner lists.

List 0 List 1
List= [[‘a’ ,’b’ ,’c’] , [1 , 2, 3]]
Main list

print (List [0]) Output: ['a', 'b', 'c']
print (List [0][2]) Output: c

list= [numbers, strings]
print(list) Output: [[1, 2, 3], ['a', 'b', 'c']]

print(numbers + strings) Output: [1, 2, 3, 'a', 'b', 'c']
a=list("abc") Output: ['a', 'b', 'c']

Other operations that can be used with list:

OPERATION	**list = ['a', 'b', 'c']**	**OUTPUT**
REPLACE	list[2]= 'd'	['a', 'b', 'd']
DELETE	del list[1]	['a', 'c']
INSERT	list.insert(2, 'm')	['m', 'a', 'b', 'c']
APPEND (Add to end list)	list.append('l')	['a', 'b', 'c', 'l']
REVERSE	list.reverse()	['c', 'b', 'a']
.JOIN	joined_string = ' - '.join(my_list)	a - b - c
OPERATION	**list = [4, 1, 2]**	**OUTPUT**
SORT	list .sort()	[1, 2, 4]

TUPLES

Tuples in Python serve as immutable data structures, meaning once created, their elements cannot be modified, removed, appended, or inserted. The immutability of tuples ensures data integrity by preventing unintentional changes, thus adding an extra layer of protection against alterations or modifications.
Tuples are valuable due to their performance benefits, offering faster processing compared to lists.

Example:

```
list=(0,1,1,2,3)
print(list[3])          Output: 2
```

Tuple with a single element requires a trailing comma (comma placed at the end of a sequence) to distinguish it from a parenthesized expression. This is because parentheses are used for grouping in Python, and a single value in parentheses without a comma is not recognized as a tuple.

```
single_element_tuple = (42,)
not_a_tuple = (42)
```

BUILT-IN FUNCTIONS

The len() function is used to determine the number of elements in an object. An object can be any data type that supports length measurement, such as strings, lists, tuples, dictionaries, and sets. This function returns an integer representing the total count of elements within the object.

It doesn't require dealing with indexing rules directly; it simply returns the number of elements in the given sequence.

Example 1:

```
list=['apple', 'orange', 'cherry']
x=len(list)
print(x)                    Output: 3 #Because there are three elements in the list.
print(list[0])              Output: apple

greeting= 'GOOD DAY!'
x=len(greeting)
print(x)                    Output: 9 #Nine elements in the list including space.

my_list = [1, 2, 3, 4, 5]
length = len(my_list)
print(length)               Output: 5
```

Example 2:

```
grades = [95.5, 89.2, 75.8, 88.0, 91.3]
#Calculating the average
average_grade = sum(grades) / len(grades)
#Displaying the results
print("Average Grade:", format(average_grade,'.2f'))      Output: Average Grade: 87.96
```

Other built in function that can be used in Python:

FUNCTIONS	**list = 3,2,5**	
min()	print(min(list))	2
max()	print(max(list))	5
sorted()	print(sorted(list))	[2, 3, 5]
sum()	print(sum(list))	10

range() function in Python is a built-in function used to generate a series of numbers. It is most used for looping a specific number of times in a **for** loop.

range(stop) creates a sequence of numbers from 0 to stop - 1.
range(start, stop, step) creates a sequence of numbers from start to stop - 1, incrementing by step. If step is not given, it defaults to 1.

start (optional): An integer specifying at which position to start. Default is 0.
stop: An integer specifying at which position to stop (not included in the sequence).

step (optional): An integer specifying the incrimination. Default is 1. A negative number makes the range count down.

EXAMPLES:

```
for i in range(3):
    print(i)
```

Output:0
1
2

```
for i in range(0, 6, 2):
    print(i)
```

>>>0
2
4

```
for i in range(1, 4):
    print(i)
```

Output:1
2
3

FUNCTION

A function is a named block of code that performs a specific task or set of tasks. Functions help in reducing code duplication by letting you write the piece of code once and calling it whenever needed. Additionally, functions can take in data, known as parameters, and return data as a result. This enhances code modularity and maintainability, making programs more organized and easier to understand.

Parameters are variables used in a function to represent the data that the function will operate on. When you call the function, you provide a value for that parameter, and these values are referred to as arguments. Arguments are the actual data that gets passed into the function when it is called. This allows the function to work with different sets of data each time it is called.

Example:

```
def my_function():
    print("This is a function")

my_function() #Outside function.
```

def = Used to declare a function.

my_function = Name of the function.

() = Brackets are used to define parameters for the function. In this case, empty parentheses mean the function doesn't take any parameters. Parameters are used to specify what data or information the function needs to operate with. When calling the function, you still use () to indicate that it's a function call in this case.

: = Colon indicates the start of the function body.

my_function() = Call the function, essentially triggering the execution of the set of instructions within that function

The function body is indented. Everything indented under the function is part of the function.

Example 1:

```
def add(a, b):
    result = a + b
    return result

sum_result = add(3, 4)
print("Sum:", sum_result)
```

In EXAMPLE 2, the **add** function takes two parameters (a and b), calculates their sum, and returns the result. The returned value is then stored in the variable sum_result and printed.

The **return** statement signals the completion of the function's task and indicates that the result of the addition operation (result) should be sent back to the code that called the function.

Example 2:

#Function enters an infinite loop with while True, prompt the user to enter a number until a valid integer is provided.

#Within the loop, it prompts the user to enter a number and stores the input in the variable user_input.
It then attempts to convert the user input to an integer, and stores it in a new variable called number.
If the conversion is successful, the function returns the value in the variable number.
If the conversion raises a ValueError (i.e., if the input is not a valid integer), it catches the exception, prints an error message indicating that the input is invalid, and the loop continues, prompting the user to enter another input.

```
def get_valid_input():
    while True:
        user_input = input("Enter Number: ")
        try:
            number = int(user_input)
            return number
        except ValueError:
            print("Invalid input. Please enter a valid integer.")
```

#Function to check if a number is divisible by two, three and to calculate the square of the number.

```
def test_number_properties(test_number):

    #Test divisibility by two
    if test_number % 2 == 0:
        print(f"{test_number} is divisible by 2.")
    else:
        print(f"{test_number} is not divisible by 2.")

    #Test divisibility by three
    if test_number % 3 == 0:
        print(f"{test_number} is divisible by 3.")
    else:
        print(f"{test_number} is not divisible by 3.")

    #Calculate and print the square of the number
    square_result = test_number ** 2
    print(f"The square of {test_number} is {square_result} ")

def main():
```

#The first loop, also known as the outer loop, guarantees the program's continuous operation, enabling users to test various numbers properties without necessitating a restart of the application.

```
while True:
        test_number = get_valid_input()
```

```
        test_number_properties(test_number)
#The second loop, also referred to as the nested loop, operates continuously until it receives a valid response ('yes' or 'no') from the user. Upon receiving 'yes', the break statement is executed, terminating the nested loop and subsequently resuming the execution of the outer loop. If 'no' is inputted, a return statement is executed, leading to the termination of the main function.
        while True:
            another_test = input("Do you want to test another number? (yes/no): ").lower()

            if another_test == 'no':
                return
            elif another_test == 'yes':
                break
            else:
                print("Invalid input. Please enter 'yes' or 'no'.")

#Call the main function so that it can run.
if __name__ == "__main__":
    main()
```

IMPORTING FUNCTIONS

In modular programming, code is divided into separate modules or files, each focused on specific tasks. This improves code maintenance and clarity. For example, functions defined in one Python file can be used in another, enhancing code organization and reusability.

HOW TO IMPORT:
Create a Python file with your functions, in this case calling it functions.py

```
#This is my functions.py file.
def greet(name):
    return f"Hello, {name}!"
def square(num):
    return num ** 2
```

Importing All Names:
This method uses the syntax from module_name import *. It imports all public names defined in the module into the current namespace. However, it's generally not recommended for large projects or when you're working with multiple modules, as it can lead to conflicts between names defined in different modules. It can also make your code less readable since it's not clear which module a name comes from.

EXAMPLE:
from functions import *
After this, you can directly use greet(name) and square(num) in your code.

Importing Specific Names:
Can import specific names from a module, which could be functions, variables, classes, etc. This method is clearer than importing all names, as it clearly defines which names you're going to use. This improves readability and helps to avoid name conflicts.

EXAMPLE:
from functions import greet, square

Importing an entire module:
When you import an entire module, you're bringing everything defined in that module into your code under the module's namespace. This means you'll access the module's names by prefixing them with the module name and a dot. This method is very clear and prevents naming conflicts.

EXAMPLE:
import functions
To use the functions you defined, you'll prefix them with functions., like functions.greet(name) and functions.square(num).

This approach is very tidy and makes it clear where each function comes from, which is especially useful in large projects or when using functions from multiple modules that might have overlapping names.

ERROR HANDLING

Error handling is a programming practice that involves identifying, managing, and responding to unexpected issues or errors that may occur during the execution of a software.

EXAMPLE:

```
#Initiates an infinite loop, meaning it will continue to run until explicitly terminated with a break statement.
while True:
    #Get user input for age
    age_input = input("Enter your age (or 'exit' to end): ")
    #Check if the user wants to exit
    if age_input.lower() == 'exit':
        print("Exiting the program. Goodbye!")
        #Ends the loop. Exiting the program.
        break
    try:
        age = int(age_input)
        if age < 0:
            print("Error: Age cannot be negative.")
        else:
            print(f"Your age is: {age}")
    except ValueError:
        print("Error: Please enter a valid integer for age.")
```

The **try** block is where you place code that you anticipate might throw an error during execution. The goal is to catch any exceptions that may occur and handle them gracefully instead of letting the program crash. When Python executes the code inside the try block, it keeps an eye out for any exceptions that are raised. If an exception occurs, Python stops executing the code within the try block and looks for an except block to handle the exception.

The **except** block is used to decide what to do when an error happens. You can have multiple except blocks under a single try block, each designed to catch and handle different kinds of exceptions. This way, your program knows what to do with different mistakes, helping it to keep running smoothly instead of crashing.

FILE HANDLING

File handling in Python refers to the ability of a program to interact with files on a computer. It involves operations like opening, reading, writing files.
Python's ability to handle input and output operations allows developers to create flexible applications that can efficiently store, fetch, and modify data.

Opening a File

The **WITH** statement is utilized to open a file while ensuring that the file is automatically closed when the statement block is exited, preventing resource leaks, data loss, and file locking issues.

MODE. Specify the purpose for which the file is opened.

with open("example.txt", "r") as file: ← Colon used to indicate the beginning of a code block.

Name of the file to open.

```
file_path = 'example.txt'
with open(file_path, 'r') as file
```

In the example above the file path is stored in the file_path variable. This makes the code more readable and allows for easy modification of the file path if needed.

Reading from a File

"r": Read mode. It allows reading the contents of a file. When a file is opened in read mode, you can read its contents, but you cannot modify it. If the file does not exist, attempting to open it in read mode will result in an error.

EXAMPLE 1:

```
#Open the file in read mode
with open('a.txt', 'r') as file:
    #Read the entire content of the file
    file_content = file.read()
#Print the content of the file
print(file_content)
```

.read() reads and returns the entire contents of the file.
To read a specific number of characters, use read(n), n specifies the number of characters you want to read.

EXAMPLE 2:

Can directly iterate over a file object to read it line by line. This is memory efficient and fast, making it a preferred way to read lines from a file, especially for large files.

```
#Open the file in read mode ('r')
with open('example.txt', 'r') as file:
    #Iterate over each line in the file
    for line in file:
        print(line)
```

EXAMPLE 3:
To read lines into a list, use readlines(); It is used to read all the lines from a file and return them as a list. Each element of the list corresponds to a line from the file.

```
with open('a.txt', 'r') as file:
    lines = file.readlines()
    #This will print the content of a file as a list.
    print(lines)
```

Writing to a File

Allows information to be stored and retrieved even after a program's execution ends. This is particularly useful for saving user preferences, configuration settings, or any information that you want to keep or maintain even after the current use or session of your application or program ends.
When a file is opened in write mode ('w'), it will remove any existing content of that file, and write the new data.

EXAMPLE:

```
#File path variable
file_path = "example.txt"
#Data to be written to the file
data_to_write = "Hello, this is an example."
#Open the file in 'w' mode (write mode)
with open(file_path, "w") as file:
    #Write the data to the file, will delete everything existing in it.
    file.write(data_to_write)
```

Appending to a File

Appending to a file involves adding new data to the end of an existing file without removing or overwriting its current content.

EXAMPLE:

```
file_path = "example.txt"
#Data to be appended to the file
data_to_append = "This is additional content."
#Open the file in 'a' mode (append mode)
with open(file_path, "a") as file:
    #Append the new data to the file
    file.write(data_to_append)
```